

Maxxes PostgreSQL Database Schema Document

Updated to Match Week 5 Canonical Data Model

1. Purpose

This document defines the PostgreSQL database schema for the **Maxxes Fan Loyalty & Intelligence Platform** based on the Week 5 complete build document. The Week 5 document identifies **9 canonical entities** as the official data model for Maxxes. This schema follows those entities directly so that the backend, frontend, design, QA, and documentation teams all reference the same structure.

Maxxes is a fan loyalty and intelligence platform that uses cross-platform fan behavior signals to calculate XP, assign tiers, rank fans, unlock rewards, and support business intelligence for venues, artists, managers, and sponsors.

2. Database Technology

Recommended Database

PostgreSQL

Why PostgreSQL Fits Maxxes

PostgreSQL is the best fit because Maxxes depends on structured relationships between fans, fan events, XP transactions, challenges, rewards, campaigns, and venue summary analytics. The app needs ranking, filtering, aggregation, and historical behavior tracking, which are all strong use cases for a relational database.

For the first MVP, the team can still use mock JSON data or seed files. However, the long-term backend structure should follow this PostgreSQL schema.

3. Official Week 5 Canonical Entities

The Week 5 document identifies these 9 entities as the **single source of truth** for the Maxxes data model:

1. Fan
2. FanEvent
3. XPTransaction

4. Challenge
5. ChallengeProgress
6. Reward
7. RewardRedemption
8. Campaign
9. VenueSummary

This document intentionally centers the database design around these 9 official entities.

4. Entity Relationship Overview

Core Relationships

- One Fan can have many FanEvents.
 - One FanEvent can generate one or more XPTransactions.
 - One Fan can participate in many Challenges through ChallengeProgress.
 - One Challenge can have many ChallengeProgress records.
 - One Fan can redeem many Rewards through RewardRedemption.
 - One Reward can have many RewardRedemption records.
 - One Campaign can be connected to many FanEvents.
 - One VenueSummary stores aggregate operator-facing analytics for a venue, artist, or event context.
-

5. PostgreSQL Schema

5.1 fans

Represents the master user record and ranking anchor for each fan.

```
CREATE TABLE fans (  
  fan_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  fan_name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) UNIQUE,  
  city VARCHAR(100),  
  preferred_team_artist VARCHAR(150),  
  current_tier VARCHAR(20) DEFAULT 'Bronze',  
  xp_total INTEGER DEFAULT 0,  
  streak_days INTEGER DEFAULT 0,  
  last_active_date DATE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```

    CONSTRAINT valid_fan_tier CHECK (
        current_tier IN ('Bronze', 'Silver', 'Gold', 'Platinum', 'Diamond')
    )
);

```

Week 5 Alignment

The Week 5 document lists Fan fields such as `fan_id`, `city`, `preferred_team_artist`, `current_tier`, `xp_total`, and `streak_days`.

Purpose

This table stores the main fan profile and supports ranking, tier assignment, milestone tracking, and fan segmentation.

5.2 fan_events

Represents the immutable behavioral signal log. Every XP-earning action is recorded here.

```

CREATE TABLE fan_events (
    fan_event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    fan_id UUID NOT NULL REFERENCES fans(fan_id) ON DELETE CASCADE,
    event_type VARCHAR(50) NOT NULL,
    source_platform VARCHAR(75),
    duration_minutes INTEGER,
    amount_usd NUMERIC(10,2),
    venue_id VARCHAR(100),
    campaign_id UUID,
    timestamp TIMESTAMP NOT NULL,
    metadata JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Week 5 Alignment

The Week 5 document lists FanEvent fields such as `event_type`, `source_platform`, `duration_minutes`, `amount_usd`, `venue_id`, `campaign_id`, and `timestamp`.

Example event_type Values

- verified_attendance
- watch_time
- listening_time
- ticket_purchase

- merch_purchase
- concession_purchase
- social_post
- promo_click
- qr_scan
- friend_referral
- challenge_completion
- reward_redemption

Purpose

This table stores the raw behavioral signals that become XP, tiers, rankings, segments, and recommendations.

5.3 xp_transactions

Stores the audit trail for scoring. This makes the XP model explainable and testable.

```
CREATE TABLE xp_transactions (
  xp_transaction_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  fan_id UUID NOT NULL REFERENCES fans(fan_id) ON DELETE CASCADE,
  source_event_id UUID REFERENCES fan_events(fan_event_id) ON DELETE SET NULL,
  xp_earned INTEGER NOT NULL,
  multiplier NUMERIC(4,2) DEFAULT 1.00,
  xp_reason VARCHAR(150),
  calculated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Week 5 Alignment

The Week 5 document lists XPTransaction fields such as `source_event_id`, `xp_earned`, `multiplier`, `xp_reason`, and `calculated_at`.

Purpose

This table records how much XP was earned, why it was earned, and what multiplier was applied.

5.4 challenges

Defines gamified actions fans can complete for XP.

```
CREATE TABLE challenges (
  challenge_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  challenge_name VARCHAR(150) NOT NULL,
  challenge_type VARCHAR(75),
  required_action VARCHAR(150),
  xp_reward INTEGER NOT NULL,
  sponsor_id VARCHAR(100),
  active_start TIMESTAMP,
  active_end TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Week 5 Alignment

The Week 5 document lists Challenge fields such as `challenge_type`, `required_action`, `xp_reward`, `sponsor_id`, and `active_window`.

Example challenge_type Values

- attendance_streak
- watch_time_goal
- listening_goal
- qr_scan
- social_share
- referral
- purchase_goal

Purpose

This table defines the actions that drive gamification and repeat fan engagement.

5.5 challenge_progress

Tracks each fan's progress toward completing a challenge.

```
CREATE TABLE challenge_progress (
  challenge_progress_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  fan_id UUID NOT NULL REFERENCES fans(fan_id) ON DELETE CASCADE,
  challenge_id UUID NOT NULL REFERENCES challenges(challenge_id) ON DELETE CASCADE,
  current_progress INTEGER DEFAULT 0,
  target_progress INTEGER NOT NULL,
  status VARCHAR(30) DEFAULT 'in_progress',
  completed_at TIMESTAMP,
```

```

    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT valid_challenge_progress_status CHECK (
        status IN ('in_progress', 'completed', 'expired')
    )
);

```

Week 5 Alignment

The Week 5 document lists ChallengeProgress fields such as `fan_id`, `challenge_id`, `current_progress`, `target_progress`, and `completed_at`.

Purpose

This table allows the Fan Home / My Journey screen to display progress toward XP-based goals.

5.6 rewards

Defines redeemable experiences and eligibility requirements.

```

CREATE TABLE rewards (
    reward_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    reward_name VARCHAR(150) NOT NULL,
    reward_category VARCHAR(75),
    tier_required VARCHAR(20),
    xp_cost INTEGER,
    quantity_available INTEGER,
    partner_id VARCHAR(100),
    active_start TIMESTAMP,
    active_end TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT valid_reward_tier CHECK (
        tier_required IN ('Bronze', 'Silver', 'Gold', 'Platinum', 'Diamond')
    )
);

```

Week 5 Alignment

The Week 5 document lists Reward fields such as `tier_required`, `xp_cost`, `quantity_available`, `reward_category`, and `partner_id`.

Example reward_category Values

- ticket_discount
- vip_upgrade
- early_access
- presale_access
- meet_and_greet
- merch_discount
- concession_credit
- exclusive_content

Purpose

This table powers the Rewards Marketplace and tier-gated perk logic.

5.7 reward_redemptions

Tracks reward usage and remaining catalog inventory.

```
CREATE TABLE reward_redemptions (  
  reward_redemption_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  reward_id UUID NOT NULL REFERENCES rewards(reward_id) ON DELETE CASCADE,  
  fan_id UUID NOT NULL REFERENCES fans(fan_id) ON DELETE CASCADE,  
  xp_spent INTEGER,  
  status VARCHAR(30) DEFAULT 'pending',  
  redeemed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  CONSTRAINT valid_reward_redemption_status CHECK (  
    status IN ('pending', 'redeemed', 'cancelled', 'expired')  
  )  
);
```

Week 5 Alignment

The Week 5 document lists RewardRedemption fields such as `reward_id`, `fan_id`, `xp_spent`, `status`, and `redeemed_at`.

Purpose

This table tracks which rewards were used and supports reward performance reporting.

5.8 campaigns

Supports attribution logic and campaign lift measurement.

```
CREATE TABLE campaigns (  
  campaign_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  campaign_name VARCHAR(150) NOT NULL,  
  campaign_type VARCHAR(75),  
  audience_segment VARCHAR(100),  
  offer_type VARCHAR(100),  
  channel VARCHAR(75),  
  start_date DATE,  
  end_date DATE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Week 5 Alignment

The Week 5 document lists Campaign fields such as `campaign_type`, `audience_segment`, `offer_type`, `channel`, `start_date`, and `end_date`.

Example campaign_type Values

- ticket_discount
- presale_access
- vip_offer
- reengagement
- sponsor_activation
- qr_challenge
- reward_challenge

Purpose

This table supports campaign tracking, audience segment targeting, and promotion response analytics.

5.9 venue_summaries

Stores operator-facing aggregate snapshots that power the Venue Intel Dashboard.

```
CREATE TABLE venue_summaries (  
  venue_summary_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  venue_id VARCHAR(100) NOT NULL,  
  generated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  avg_xp NUMERIC(10,2),
```



```

    tier_distribution JSONB,
    presale_conversion_rate NUMERIC(5,2),
    churn_flag_count INTEGER,
    projected_revenue NUMERIC(12,2),
    reward_redemption_rate NUMERIC(5,2),
    campaign_lift_summary JSONB,
    top_fans_snapshot JSONB
);

```

Week 5 Alignment

The Week 5 document lists VenueSummary fields such as `venue_id`, `avg_xp`, `tier_distribution`, `presale_conversion_rate`, `churn_flag_count`, and `generated_at`.

Purpose

This table stores dashboard-ready aggregate analytics, including tier distribution, presale conversion, churn flags, and top fan snapshots.

6. XP Scoring Rules

The Week 5 document makes the XP model the official scoring system for Maxxes.

Signal	XP Rule	Purpose
Verified event attendance	+200 XP per event	Strongest proof of real-world participation
Watch time	+5 XP per hour	Strong intent signal
Listening time	+3 XP per hour	Useful but more passive engagement signal
In-venue / merch spend	+1 XP per \$1	Connects engagement to revenue
Challenge completion	+100 to +1,500 XP	Drives repeat engagement
Streak modifier	1.0x–2.0x	Rewards consistency
Post-event social amplification	+10 to +50 XP	Measures fan-generated promotion
Friend recruitment	+75 XP per converted referral	Captures organic growth

7. Tier Rules

The Week 5 document defines these official Maxxes tiers.

Tier	XP Threshold	Meaning
Bronze	0–2,499	Entry-level or casual engagement
Silver	2,500–4,999	Consistent but developing loyalty
Gold	5,000–9,999	Meaningful repeat engagement
Platinum	10,000–24,999	High-value fan across multiple signal types
Diamond	25,000+	Top-tier fan with scarce access eligibility

8. RFM Overlay Logic

The XP model should include Recency, Frequency, and Monetary logic.

Recency

Recent activity should carry more weight than old activity.

Recommended rule:

- XP from the past 30 days receives a 2.0x recency weight.
- XP from 31–90 days receives a 1.0x weight.
- XP older than 90 days receives a lower weight or is used mainly for historical context.

Frequency

Streaks increase the multiplier from 1.0x to 2.0x on selected base XP.

Monetary

Spend XP should contribute to the model, but should not dominate rankings. A cap can be applied so monetary XP does not exceed 20% of the total score weight.

9. Recommended Indexes

```
CREATE INDEX idx_fans_xp_total ON fans(xp_total DESC);
CREATE INDEX idx_fans_current_tier ON fans(current_tier);
CREATE INDEX idx_fans_last_active_date ON fans(last_active_date);

CREATE INDEX idx_fan_events_fan_id ON fan_events(fan_id);
CREATE INDEX idx_fan_events_event_type ON fan_events(event_type);
CREATE INDEX idx_fan_events_campaign_id ON fan_events(campaign_id);
CREATE INDEX idx_fan_events_timestamp ON fan_events(timestamp);
CREATE INDEX idx_fan_events_venue_id ON fan_events(venue_id);

CREATE INDEX idx_xp_transactions_fan_id ON xp_transactions(fan_id);
CREATE INDEX idx_xp_transactions_source_event_id ON
xp_transactions(source_event_id);
CREATE INDEX idx_xp_transactions_calculated_at ON
xp_transactions(calculated_at);

CREATE INDEX idx_challenge_progress_fan_id ON challenge_progress(fan_id);
CREATE INDEX idx_challenge_progress_challenge_id ON
challenge_progress(challenge_id);
CREATE INDEX idx_reward_redemptions_fan_id ON reward_redemptions(fan_id);
CREATE INDEX idx_reward_redemptions_reward_id ON reward_redemptions(reward_id);
CREATE INDEX idx_campaigns_audience_segment ON campaigns(audience_segment);
CREATE INDEX idx_venue_summaries_venue_id ON venue_summaries(venue_id);
```

10. Recommended Views

Views simplify frontend queries and dashboard rendering.

10.1 fan_leaderboard_view

```
CREATE VIEW fan_leaderboard_view AS
SELECT
    fan_id,
    fan_name,
    city,
    preferred_team_artist,
    current_tier,
    xp_total,
    streak_days,
```

```
last_active_date,  
RANK() OVER (ORDER BY xp_total DESC) AS leaderboard_rank  
FROM fans  
ORDER BY xp_total DESC;
```

Supports

- Fan Leaderboard
- Ranked Access Queue
- Venue Intel Dashboard top fan list

10.2 fan_xp_breakdown_view

```
CREATE VIEW fan_xp_breakdown_view AS  
SELECT  
    f.fan_id,  
    f.fan_name,  
    fe.event_type,  
    SUM(xt.xp_earned) AS total_xp_from_signal  
FROM fans f  
JOIN xp_transactions xt ON f.fan_id = xt.fan_id  
JOIN fan_events fe ON xt.source_event_id = fe.fan_event_id  
GROUP BY f.fan_id, f.fan_name, fe.event_type;
```

Supports

- Superfan Score Card
- XP breakdown by signal type
- Business Intelligence Brief

10.3 at_risk_high_value_fans_view

```
CREATE VIEW at_risk_high_value_fans_view AS  
SELECT  
    fan_id,  
    fan_name,  
    current_tier,  
    xp_total,  
    last_active_date,  
    CURRENT_DATE - last_active_date AS inactive_days  
FROM fans
```

```
WHERE current_tier IN ('Gold', 'Platinum', 'Diamond')
AND last_active_date < CURRENT_DATE - INTERVAL '30 days';
```

Supports

- Lapse Risk Alert
- At-Risk Gold+ Fans recommendation card
- Retention Alert Panel

10.4 reward_eligibility_view

```
CREATE VIEW reward_eligibility_view AS
SELECT
    f.fan_id,
    f.fan_name,
    f.current_tier,
    f.xp_total,
    r.reward_id,
    r.reward_name,
    r.reward_category,
    r.tier_required,
    r.xp_cost,
    CASE
        WHEN f.xp_total >= r.xp_cost THEN true
        ELSE false
    END AS has_enough_xp
FROM fans f
CROSS JOIN rewards r;
```

Supports

- Rewards Marketplace
- Perk eligibility panel
- Fan Home / My Journey screen

11. Backend Logic Supported by Schema

11.1 XP Calculation Flow

1. A fan behavior is inserted into `fan_events`.
2. The backend checks `event_type`, `duration_minutes`, `amount_usd`, or `metadata`.
3. XP is calculated using the Week 5 XP rules.

4. A record is inserted into `xp_transactions`.
 5. The fan's `xp_total` is updated in `fans`.
 6. The fan's `current_tier` is recalculated.
 7. Any relevant recommendations, reward eligibility, or segment labels are updated in the frontend output.
-

11.2 Tier Assignment Flow

1. Read `fans.xp_total`.
 2. Compare against Bronze–Diamond thresholds.
 3. Update `fans.current_tier`.
 4. Frontend displays updated tier badge and milestone tracker.
-

11.3 Challenge Completion Flow

1. Fan activity updates `challenge_progress`.
 2. If `current_progress >= target_progress`, mark status as `completed`.
 3. Insert a related `fan_events` record with `event_type = challenge_completion`.
 4. Create an `xp_transactions` record using the challenge XP reward.
-

11.4 Reward Redemption Flow

1. Fan selects reward.
 2. Backend checks `fans.current_tier`, `fans.xp_total`, and `rewards.xp_cost`.
 3. If eligible, create `reward_redemptions` record.
 4. Deduct XP if the reward requires XP spend.
 5. Update reward inventory if needed.
-

11.5 Venue Intel Dashboard Flow

1. Aggregate Fan, FanEvent, XPTransaction, RewardRedemption, and Campaign data.
 2. Generate tier distribution.
 3. Calculate average XP, presale conversion, reward redemption, churn flags, and projected revenue.
 4. Store snapshot in `venue_summaries`.
 5. Frontend renders the Venue Intel Dashboard.
-

12. Dashboard Components Supported

Fan-Facing Components

- Fan Home / My Journey
- Superfan XP Score Card
- Tier Badge
- Next Milestone Tracker
- Rewards Marketplace
- Challenges Panel
- Fan Leaderboard

Operator-Facing Components

- Venue Intel Dashboard
 - Tier Distribution Chart
 - Campaign Lift by Segment
 - Lapse Risk Alert
 - Revenue Projection Panel
 - Recommendation Action Queue
 - Business Intelligence Brief
-

13. MVP Seed Data Recommendation

For Week 5–6, seed the database or mock JSON files with:

- 25+ fans across all 5 tiers
- 90 days of FanEvent history
- Multiple XPTransaction records per fan
- 5–10 challenges
- 5–10 rewards
- 3–5 campaigns
- 1–3 VenueSummary snapshots

This is enough to demonstrate the XP score card, leaderboard, rewards logic, Venue Intel dashboard, and recommendation cards.

14. MVP Assumptions

- Fan-level behavior can be mocked or simulated.
- Live integrations with Ticketmaster, Spotify, Instagram, POS, and QR systems are outside MVP scope.
- XP and tier logic are rule-based.

- Recommendation logic is rules-based, not machine learning.
 - PostgreSQL is the recommended database structure, but mock JSON can be used first for fast prototyping.
 - Authentication, payment processing, and production-grade security are outside the first MVP.
-

15. Week 5 Backend Deliverables

For Week 5, backend should deliver:

- Finalized 9-entity data dictionary
 - PostgreSQL schema draft
 - XP rules document
 - Bronze–Diamond tier thresholds
 - Mock Fan and XPTransaction dataset plan
 - Recommendation rules outline
 - Frontend data contract for score card, leaderboard, rewards, and Venue Intel dashboard
-

16. Week 6 Backend Deliverables

For Week 6, backend should deliver:

- Mock Fan and FanEvent seed data
 - XP calculation logic
 - Tier assignment logic
 - Leaderboard output
 - Reward eligibility output
 - At-risk fan output
 - Basic VenueSummary output
 - Screenshots or output examples for lab journal evidence
-

17. Summary

This schema updates the Maxxes database design to match the Week 5 document's official 9-entity data model. PostgreSQL is recommended because Maxxes depends on relational, auditable, and analytics-ready data. The structure supports the core Maxxes chain:

Fan behavior signal → FanEvent → XPTransaction → XP total → Tier → Reward eligibility → Recommendation → Venue Intelligence output

That chain is the foundation of Maxxes as a business analytics platform rather than just a fan app.